

Project Report

Course: Python Essentials

Submitted By

Amrita Kumari

Registration Number: 26BAI10387

Course: Bachelor of Technology — Computer Science & Engineering

University: VIT Bhopal University

Academic Year: 2026–2027

1. INTRODUCTION

Libraries maintain collections of books and provide those resources to students and other users. Even a small library requires a systematic method to keep track of available books, registered students, issued books, return dates, and overdue charges.

The Library Management System developed for this project provides a simple console-based solution for these basic activities. It is implemented entirely in Python and is designed around a menu-driven approach. The user selects an operation from the main menu, and the corresponding function performs the requested task.

The source code initializes three main data collections: books, students, and issued_books. The initial book collection contains three sample books, while the student and issued-book collections begin empty. Each book is represented using a dictionary containing an ID, title, author, category, and quantity.

The program is modular. Separate functions are provided for book management, student management, circulation, statistics, and utility operations. This structure makes the program easier to understand and maintain.

The application is not a database-driven or web-based system. All records are maintained in memory using Python data structures. Therefore, the project focuses primarily on demonstrating programming logic and basic library workflows rather than enterprise-level deployment.

2. PROBLEM STATEMENT

A library needs to maintain accurate information about its books and students and must be able to determine which books are available or currently issued. Manual management of these activities can become difficult as the number of records increases.

The project addresses the problem of managing basic library operations through a simple software application. The system should provide mechanisms to:

- Maintain book details such as ID, title, author, category, and quantity.
- Register student details such as ID, name, course, and phone number.
- Search for books using title, author, or category.
- Search for students using student ID or name.
- Issue available books to registered students.
- Prevent a student from issuing the same book more than once at the same time.
- Calculate a fourteen-day due date when a book is issued.
- Process book returns and calculate an overdue fine.
- Display currently issued books.
- Provide basic statistics about the library.

The implemented program meets these requirements through a menu-driven console interface. The main menu exposes twelve options, including book operations, student operations, issue and return operations, statistics, and exit.

3. OBJECTIVES OF THE PROJECT

The primary objective is to develop a simple Python-based Library Management System that demonstrates how programming concepts can be applied to a practical management problem.

3.1 Primary Objectives

- To create a menu-driven library management application.
- To manage book records using Python dictionaries and lists.
- To maintain student registration records.
- To implement book searching based on user-provided keywords.
- To implement book issue and return operations.
- To maintain book availability by changing quantity during issue and return.
- To calculate due dates using Python date-time operations.
- To calculate overdue fines based on the number of late days.
- To provide a summary of library statistics.

3.2 Learning Objectives

- Understand Python functions and modular programming.
- Use lists and dictionaries for structured data.
- Apply loops and conditional statements to solve management tasks.
- Use exception handling for invalid numerical input.
- Work with datetime and timedelta for date calculations.
- Build a simple interactive console application.

4. SCOPE OF THE PROJECT

The current scope covers the core operations required for a small console-based library management demonstration.

4.1 Included Scope

- Book addition, display, search, and removal.
- Student registration, display, and search.
- Book issue and return.
- Tracking of currently issued books.
- Fourteen-day lending period.
- ₹5 per-day overdue fine calculation.
- Basic library statistics.
- Input validation for numeric fields.

4.2 Outside the Current Scope

- Permanent database storage.
- User authentication and role management.
- Web or mobile interface.
- Email or SMS notifications.
- Book reservation or waiting-list management.

- Multiple-library or branch management.
- Barcode or QR-code scanning.
- Online payment of fines.
- Cloud deployment.

These items are not represented as existing features because they are not implemented in the supplied source code.

5. EXISTING SYSTEM

For a small academic or demonstration library, records may be maintained manually using registers, spreadsheets, or informal lists. Such approaches require users to locate and update records manually.

5.1 General Issues with Manual Management

- Book availability can be difficult to track accurately.
- Searching for a particular book can take additional time.
- Issue and return information may require manual calculations.
- Overdue fine calculation can be performed inconsistently.
- Maintaining separate records for students and books can be inconvenient.

The project uses a programmatic approach to automate these basic operations. It does not attempt to replace a full enterprise library platform; instead, it provides a focused implementation suitable for learning and small-scale demonstration.

6. PROPOSED SYSTEM

The proposed system is a Python console application that organizes library operations into separate functions. A user interacts with the application through the main menu and selects the required operation.

6.1 Core Workflow

1. Start the Python program.
2. Display the Library Management System menu.
3. Select a library operation.
4. Collect the required input.
5. Validate the input and locate relevant records.
6. Perform the selected operation.
7. Display the result.
8. Pause and return to the main menu.
9. Exit when option 12 is selected.

The main function implements a continuous while loop and maps menu choices to their corresponding functions.

7. SYSTEM REQUIREMENTS

7.1 Hardware Requirements

Component	Minimum Requirement
Processor	Any modern dual-core processor
RAM	2 GB or more
Storage	At least 100 MB free space
Display	Standard monitor
Keyboard	Required for console input

7.2 Software Requirements

Software	Requirement
Operating System	Windows / Linux / macOS
Programming Language	Python 3.x
Python Modules	os, datetime
Interface	Command-line / terminal
Database	Not required; data is stored in memory

The source code imports os and datetime/timedelta. The os module is used for clearing the terminal, while datetime and timedelta are used for issue, due-date, return, and overdue calculations.

8. TECHNOLOGIES USED

8.1 Python

Python is the primary programming language used to implement the system. The project uses basic Python syntax, functions, collections, loops, conditional statements, exception handling, and date-time operations.

8.2 Lists

Lists are used to store collections of books, students, and issued-book records. The books and students collections can grow through append operations, and records can be removed when necessary.

8.3 Dictionaries

Each book, student, and issue record is represented as a dictionary. Dictionary keys provide meaningful names for individual fields.

8.4 datetime and timedelta

The datetime module is used to obtain the current date and time. timedelta is used to add fourteen days to the issue date. During return, datetime.strptime converts the stored due-date string back into a date object so that overdue days can be calculated.

8.5 os Module

The os module is used by clear_screen() to clear the terminal using cls on Windows and clear on other operating systems.

9. SYSTEM ANALYSIS

9.1 Functional Requirements

ID	Function	Requirement
FR-01	Add Book	Allow the user to add a book with ID, title, author, category, and quantity.
FR-02	Display Books	Display all currently stored books.
FR-03	Search Book	Search by title, author, or category.
FR-04	Remove Book	Remove a book if it exists and is not currently issued.
FR-05	Add Student	Register a student with ID, name, course, and phone.
FR-06	Display Students	Display all registered students.
FR-07	Search Student	Search by student ID or name.
FR-08	Issue Book	Issue an available book to a registered student.
FR-09	Return Book	Return an issued book and calculate an overdue fine.
FR-10	Issued Books	Display currently issued books and due dates.
FR-11	Statistics	Display basic library counts.
FR-12	Exit	Terminate the application.

9.2 Non-Functional Requirements

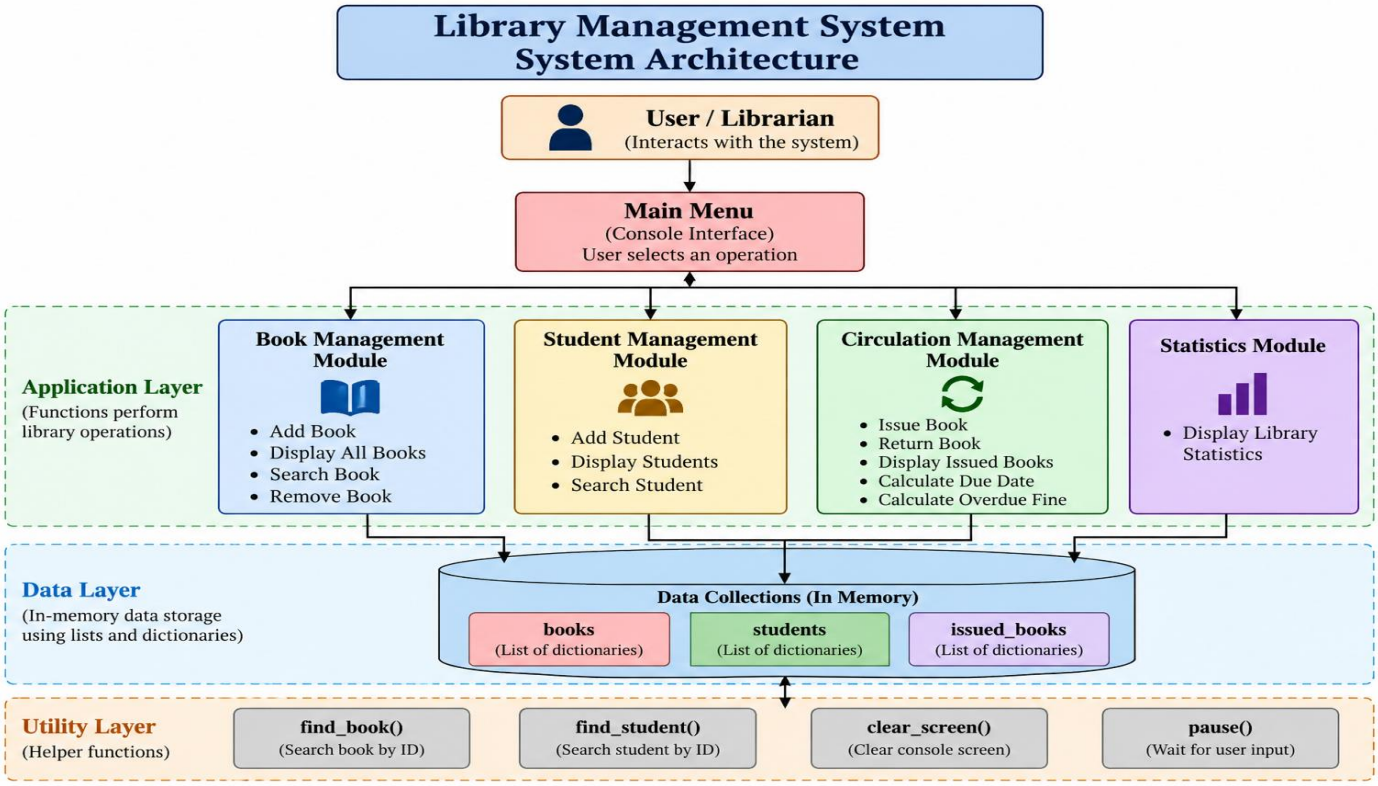
- Usability: The menu and prompts should be understandable to a basic computer user.
- Reliability: The program should handle invalid numeric input without terminating unexpectedly.
- Maintainability: Function-based organization should make individual operations easier to modify.
- Portability: The clear-screen function supports Windows and other operating systems.
- Performance: For the small in-memory data set, operations are simple and responsive.

10. SYSTEM DESIGN

10.1 High-Level Architecture

The application follows a simple procedural and function-oriented architecture. The main menu acts as the controller. It calls the selected function, while the global in-memory collections act as the application's data layer.

Layer	Implementation
Presentation	Console menu, prompts, formatted print statements
Application Logic	Python functions for books, students, circulation, and statistics
Data Layer	books, students, and issued_books lists
Utility Layer	clear_screen(), pause(), find_book(), find_student()



10.2 System Workflow

10. Program starts and executes main().
11. main() displays the menu.
12. User selects one of twelve options.
13. The selected function validates input and performs the operation.
14. The function displays a result message.
15. pause() waits for user input before returning to the menu.
16. The loop continues until the user chooses Exit.

10.3 Book Circulation Workflow

17. User enters Student ID and Book ID.
18. System searches for the student and book.
19. System verifies that both records exist.
20. System checks that quantity is greater than zero.
21. System checks whether the same student already has the same book.
22. System creates an issue record.
23. System sets the due date fourteen days after the issue date.
24. System decreases book quantity by one.

11. DATA STRUCTURES USED

11.1 Book Structure

Books are stored in the books list. Each book dictionary contains the fields id, title, author, category, and quantity. The initial source data includes three sample records.

Field	Type	Purpose
Id	Integer	Unique book identifier
Title	String	Book title
Author	String	Author name
Category	String	Book category
Quantity	Integer	Available copies

11.2 Student Structure

Student records are stored in the students list. Each record contains an ID, name, course, and phone number.

Field	Type	Purpose
Id	Integer	Unique student identifier
Name	String	Student name
Course	String	Student course
Phone	String	Student phone number

11.3 Issued Book Structure

The issued_books list stores circulation records. Each issue record contains student_id, book_id, issue_date, and due_date.

Field	Type	Purpose
student_id	Integer	Student receiving the book
book_id	Integer	Issued book identifier
issue_date	String	Date on which the book was issued
due_date	String	Calculated return deadline

12. SYSTEM MODULES

12.1 Book Management Module

The book management module contains functions for adding, displaying, searching, and removing books.

12.1.1 Add Book

add_book() asks for a book ID, title, author, category, and quantity. It first checks whether the ID already exists using find_book(). Quantity must be greater than zero. A valid record is appended to books. □filecite□turn0file0□L61-L98□

12.1.2 Display Books

display_books() prints all books in a formatted table containing ID, title, author, category, and quantity. Long text is truncated for alignment.

12.1.3 Search Book

search_book() accepts a keyword and performs a case-insensitive search against the book title, author, and category. Matching records are displayed with their available quantity.

12.1.4 Remove Book

remove_book() searches for a book by ID. If the book does not exist, a message is shown. A book cannot be removed if an active issue record exists for that book. Otherwise, it is removed from the books list.

12.2 STUDENT MANAGEMENT MODULE

12.2.1 Add Student

add_student() collects a student ID, name, course, and phone number. Duplicate student IDs are rejected. Valid records are appended to the students list.

12.2.2 Display Students

display_students() presents registered students in a formatted table containing ID, name, course, and phone number.

12.2.3 Search Student

search_student() searches using the student ID or student name. The ID is converted to a string so that a text keyword can be compared with it.

12.3 BOOK ISSUE MODULE

The issue_book() function manages the lending operation. The user supplies a Student ID and Book ID. The system searches both collections and stops the operation if either record is missing.

The function then checks whether the book has an available quantity. It also scans issued_books to determine whether the same student already has the same book. If the checks pass, the current date is obtained using datetime.now(), and the due date is calculated by adding timedelta(days=14).

An issue record is appended to issued_books, and the book quantity is reduced by one. The program then prints the issue and due dates.

12.4 BOOK RETURN MODULE

The return_book() function receives a Student ID and Book ID and searches the issued_books list for a matching record. If no record is found, the operation stops.

When a matching record is found, the corresponding book quantity is increased by one. The stored due date is parsed using datetime.strptime(). The number of overdue days is calculated by subtracting the due date from the current date. If overdue_days is greater than zero, the fine is calculated as overdue_days multiplied by 5. The issue record is then removed.

12.5 ISSUED BOOKS MODULE

display_issued_books() displays currently active issue records. For each issue record, the function finds the associated student and book and displays Student ID, Student Name, Book ID, Book Title, and Due Date.

If a referenced student or book cannot be found, the display logic uses the text “Unknown” rather than terminating.

12.6 LIBRARY STATISTICS MODULE

library_statistics() provides four basic metrics:

- Different Books – number of book records in books.
- Available Copies – sum of the quantity field across books.
- Issued Books – number of records in issued_books.
- Registered Users – number of records in students.

The function calculates these values using Python sum() and len() and displays them in a formatted console screen.

12.7 MAIN MENU MODULE

The main() function is the entry point of the application. It repeatedly clears the screen, prints the main menu, accepts a choice, and calls the corresponding function.

Option	Operation	Function
1	Add Book	add_book()
2	Display All Books	display_books()
3	Search Book	search_book()
4	Remove Book	remove_book()
5	Add Student	add_student()
6	Display Students	display_students()
7	Search Student	search_student()
8	Issue Book	Issue_book()
9	Return Book	return_book()
10	Display Issued Books	display_issued_books()
11	Library Statistics	library_statistics()
12	Exit	Terminate loop

The source code explicitly maps the twelve choices to their respective operations and prints an invalid-choice message for other input.

13. ALGORITHMS AND PROGRAM FLOW

13.1 Add Book Algorithm

25. Start add_book().
26. Read Book ID.
27. Search for an existing book with the same ID.
28. If the ID exists, display an error and stop.
29. Read title, author, category, and quantity.
30. Check whether quantity is greater than zero.
31. If invalid, display an error and stop.
32. Create a dictionary for the book.
33. Append the dictionary to books.
34. Display success message.

13.2 Search Book Algorithm

35. Read a keyword and convert it to lowercase.
36. Set found to False.
37. For each book, compare the keyword with title, author, and category.
38. If any field matches, display the book and set found to True.
39. If found remains False, display a no-match message.

13.3 Add Student Algorithm

40. Read Student ID.
41. Check whether the ID already exists.
42. If it exists, stop.
43. Read student name, course, and phone.
44. Create the student dictionary.
45. Append it to students.
46. Display success message.

13.4 ISSUE BOOK ALGORITHM

47. Read Student ID and Book ID.
48. Find the student record.
49. Find the book record.
50. If the student does not exist, stop.
51. If the book does not exist, stop.
52. If quantity is zero or less, stop.
53. Check issued_books for the same student-book combination.
54. If already issued, stop.
55. Set issue_date to the current date.
56. Set due_date to issue_date + 14 days.
57. Create and append the issue record.
58. Decrease the book quantity by one.
59. Display issue and due dates.

13.5 RETURN BOOK ALGORITHM

60. Read Student ID and Book ID.
61. Search issued_books for the matching record.
62. If no record exists, stop.
63. Find the corresponding book.
64. Increase available quantity by one.
65. Convert the stored due date to a date object.
66. Calculate overdue days.
67. If overdue days are positive, calculate fine = overdue days × 5.
68. Otherwise, set fine to zero.
69. Remove the issue record.
70. Display return status and fine.

13.6 STATISTICS ALGORITHM

71. Count the number of book records.
72. Sum available quantities.
73. Count issued-book records.
74. Count registered student records.
75. Display all four values.

14. DETAILED CODE EXPLANATION

14.1 Import Statements

The first lines import os and datetime/timedelta. The os import supports platform-specific console clearing. datetime and timedelta support date calculations for book lending.

14.2 Initial Data

The books list is initialized with three dictionaries. The examples include Python Programming, Data Structures, and Operating Systems, each with an ID, author, category, and quantity. students and issued_books are initialized as empty lists. □filecite□turn0file0□L1-L32□

14.3 Utility Functions

clear_screen() calls os.system() with cls for Windows and clear for other operating systems. pause() waits for the user to press Enter. find_book() loops through books and returns the dictionary whose id matches the supplied value. find_student() performs the equivalent operation for students. □filecite□turn0file0□L35-L57□

14.4 Book Management Code

Book management is implemented using four functions. These functions perform input collection, validation, list traversal, dictionary creation, searching, and deletion. The implementation uses simple linear searches through the books list.

14.5 Student Management Code

Student management follows a similar pattern. Student records are stored as dictionaries and accessed through linear traversal of the students list.

14.6 Circulation Code

The circulation subsystem uses issued_books to connect a student and a book. The issue record stores dates as strings in YYYY-MM-DD format. This format is later parsed during return processing.

14.7 Date Processing

At issue time, issue_date is obtained from datetime.now(). due_date is calculated using timedelta(days=14). At return time, the stored due date is parsed using datetime.strptime(record["due_date"], "%Y-%m-%d").

14.8 Main Control Flow

The main function is protected by if __name__ == "__main__":, so the menu is started when the file is executed directly. The function contains an infinite loop that ends only when the user selects option 12. □filecite□turn0file0□L547-L550□

15. INPUT VALIDATION AND ERROR HANDLING

The application uses try-except blocks around operations that convert user input to integers. If a non-numeric value is entered where an integer is required, ValueError is caught and an appropriate message is displayed.

15.1 Book Validation

- Book IDs are converted to integers.
- Duplicate Book IDs are rejected.
- Quantity must be greater than zero.
- Invalid numeric input produces an error message.

15.2 Student Validation

- Student IDs are converted to integers.
- Duplicate Student IDs are rejected.
- Invalid numeric input produces an error message.

15.3 Circulation Validation

- Student existence is checked before issue.
- Book existence is checked before issue.
- Book availability is checked before issue.
- Duplicate active issue for the same student-book combination is prevented.
- Return requires a matching active issue record.

The validation is implemented directly inside the relevant functions rather than through a separate validation class or framework.

16. FINE CALCULATION MECHANISM

The system provides a simple overdue fine calculation. Every issued book receives a due date fourteen days after the issue date.

16.1 Formula

Fine = Number of Overdue Days × ₹5

For example, if a book is returned three days after its due date, the program calculates the fine as 3 × 5 = ₹15. This is an example of the implemented formula and not a separate stored financial record.

16.2 Implementation Logic

76. Read the due date from the issue record.
77. Convert it to a datetime object.
78. Obtain the current date and time.
79. Subtract the due date from the return date.
80. Read the .days value.
81. If the value is greater than zero, multiply it by 5.
82. Otherwise, display a zero fine.

The exact implementation uses overdue_days = (return_date - due_date).days and fine = overdue_days * 5 when overdue_days is greater than zero. □filecite□turn0file0□L378-L395□

17. TESTING AND TEST CASES

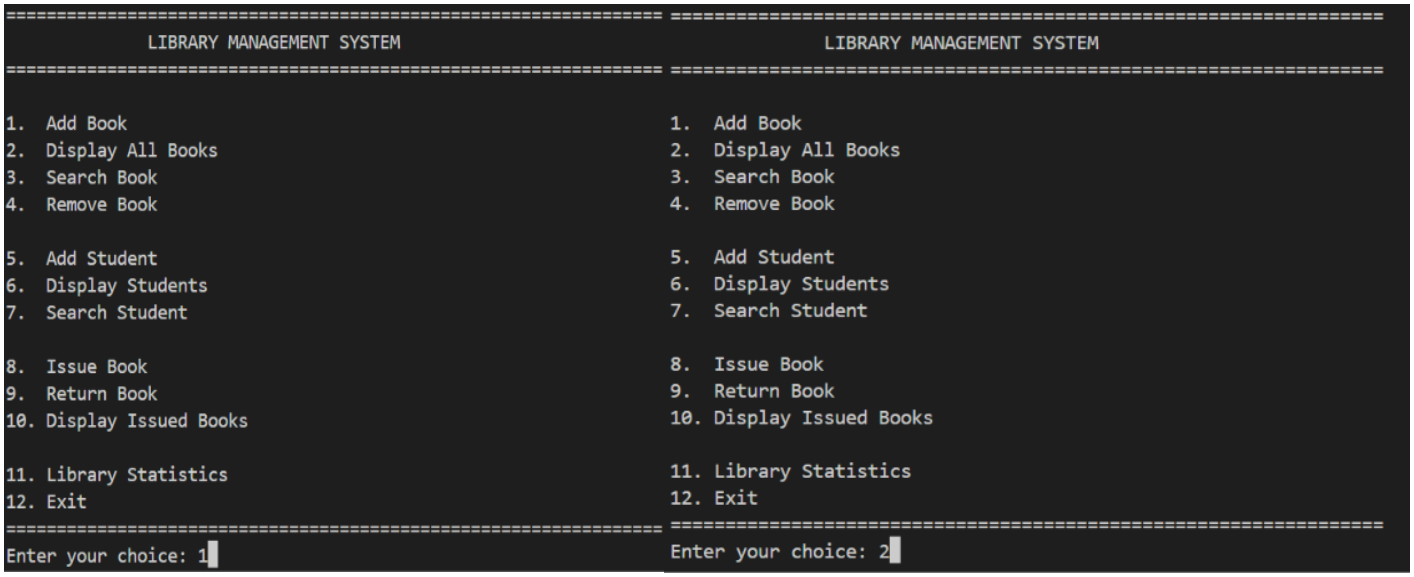
Testing was designed around the twelve operations exposed by the main menu and the validation conditions implemented in the source code.

Test ID	Test	Input / Condition	Expected Result
TC-01	Add valid book	Enter a new unique ID and positive quantity	Book added successfully
TC-02	Duplicate book ID	Use an existing book ID	Book ID already exists
TC-03	Invalid quantity	Enter quantity 0 or negative	Quantity must be greater than 0
TC-04	Search book	Enter title/author/category keyword	Matching book displayed
TC-05	Remove issued book	Remove a book with active issue	Removal prevented
TC-06	Add student	Enter unique student ID and details	Student added successfully
TC-07	Duplicate student ID	Use existing student ID	Duplicate rejected
TC-08	Issue unavailable book	Select book with quantity 0	Book unavailable
TC-09	Duplicate issue	Same student issues same book twice	Second issue prevented
TC-10	Return book	Return matching active issue	Quantity restored
TC-11	Late return	Return after due date	Fine calculated
TC-12	Invalid menu choice	Enter unsupported option	Invalid choice displayed

17.1 Testing Observations

The test cases correspond directly to conditions present in the code. Because the application stores data in memory, tests should be performed within the same execution session when they depend on records created earlier.

18. OUTPUT


LIBRARY MANAGEMENT SYSTEM
1. Add Book
2. Display All Books
3. Search Book
4. Remove Book
5. Add Student
6. Display Students
7. Search Student
8. Issue Book
9. Return Book
10. Display Issued Books
11. Library Statistics
12. Exit
Enter your choice: 1
LIBRARY MANAGEMENT SYSTEM
1. Add Book
2. Display All Books
3. Search Book
4. Remove Book
5. Add Student
6. Display Students
7. Search Student
8. Issue Book
9. Return Book
10. Display Issued Books
11. Library Statistics
12. Exit
Enter your choice: 2

```
============================================================
                      ADD NEW BOOK
============================================================
Enter Book ID: 1234
Enter Book Title: CSE
Enter Author Name: AGARWAL
Enter Category: MAIN
Enter Quantity: 5

Book added successfully!

Press Enter to continue...
```

```
================================================================================
                                ALL BOOKS
================================================================================
ID      Title                 Author              Category            Qty
--------------------------------------------------------------------------------
101     Python Programming    Mark Lutz           Programming         5
102     Data Structures       Seymour Lipschutz   Computer Science    3
103     Operating Systems     Galvin              Computer Science    4
1234    CSE                   AGARWAL             MAIN                5

Press Enter to continue...
```

```
============================================================
              LIBRARY MANAGEMENT SYSTEM
============================================================

1.  Add Book
2.  Display All Books
3.  Search Book
4.  Remove Book

5.  Add Student
6.  Display Students
7.  Search Student

8.  Issue Book
9.  Return Book
10. Display Issued Books

11. Library Statistics
12. Exit
============================================================
Enter your choice: 3
```

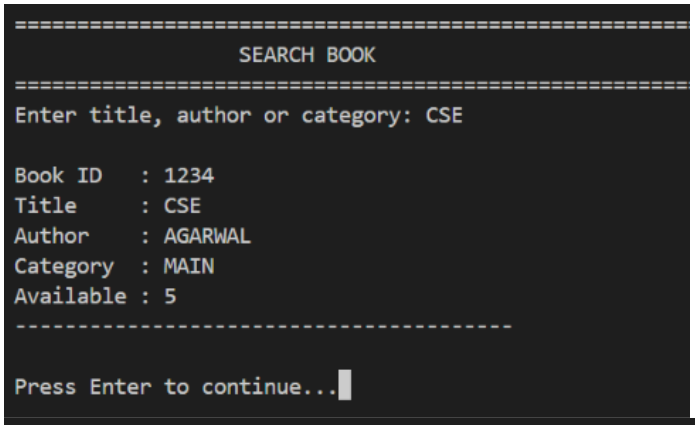
```
============================================================
              LIBRARY MANAGEMENT SYSTEM
============================================================

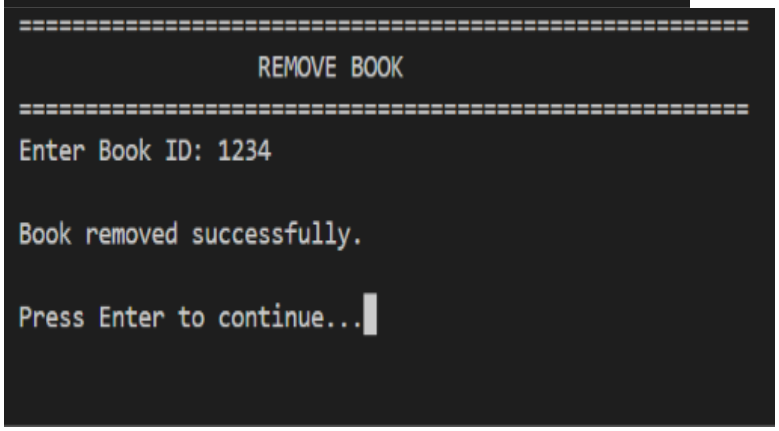
1.  Add Book
2.  Display All Books
3.  Search Book
4.  Remove Book

5.  Add Student
6.  Display Students
7.  Search Student

8.  Issue Book
9.  Return Book
10. Display Issued Books

11. Library Statistics
12. Exit
============================================================
Enter your choice: 4
```





.

19. ADVANTAGES

- Simple menu-driven interface.
- Easy-to-understand Python implementation.
- Separate functions improve code organization.
- Book searching supports title, author, and category.
- Student searching supports ID and name.
- Book availability is automatically adjusted during issue and return.
- Duplicate book and student IDs are checked.
- Duplicate active issue of the same book by the same student is prevented.
- Due dates are calculated automatically.
- Overdue fines are calculated automatically.
- Basic library statistics are available.

These advantages describe functionality that is present in the supplied implementation.

20. LIMITATIONS

- Data is stored only in memory and is lost when the program terminates.
- There is no SQLite, MySQL, PostgreSQL, or other persistent database.
- There is no login or authentication system.
- The interface is console-based.
- There is no graphical user interface.
- There is no web or mobile client.
- There is no automated notification for overdue books.
- Fine records are calculated during return but are not stored as a separate transaction history.
- There is no separate history of previously returned books.
- There is no book reservation or waiting-list feature.
- The application is intended for small-scale academic demonstration rather than multi-user deployment.

These limitations follow from the data structures and functions present in the submitted code. The project can be extended without changing the basic conceptual workflow.

21. FUTURE ENHANCEMENTS

The following improvements could be implemented in future versions:

- Add SQLite or another database for permanent storage.
- Introduce user authentication and administrator roles.
- Create a graphical user interface using Tkinter or another GUI framework.
- Develop a web interface using Flask or Django.
- Store complete transaction history for issued and returned books.
- Add book reservation and waiting-list management.
- Add automated overdue notifications.
- Add barcode or QR-code scanning.
- Store and report fine payment history.
- Add export to CSV or PDF.
- Add dashboard charts for library usage.
- Support multiple library branches and inventory locations.

These are proposed enhancements only; they are not features of the current implementation.

22. CONCLUSION

The Library Management System successfully demonstrates how Python can be used to implement a practical console-based management application. The system organizes book records, student records, and active issue records using lists and dictionaries.

The program provides the core operations required for the project: adding, displaying, searching, and removing books; adding, displaying, and searching students; issuing and returning books; displaying active issues; calculating overdue fines; and presenting library statistics.

The project also demonstrates several important programming concepts, including modular functions, loops, conditional logic, exception handling, list and dictionary operations, and date-time calculations.

Although the current implementation is limited by in-memory storage and a console-only interface, it provides a clear foundation for future development into a database-backed, multi-user library application.

23. REFERENCES

- Python Documentation – Python Standard Library documentation.
- Python datetime module documentation – for datetime and timedelta concepts.
- Python os module documentation – for operating-system interface operations.
- Submitted Library Management System Python source code – project implementation.

The project documentation is based primarily on the supplied source code. External references are included only as general technical references for the Python modules used.

24. APPENDIX – SOURCE CODE STRUCTURE

```python
import os
from datetime import datetime, timedelta
# ------------------- DATA -------------------
books = [
    {
        "id": 101,
        "title": "Python Programming",
        "author": "Mark Lutz",
        "category": "Programming",
        "quantity": 5
    },
    {
        "id": 102,
        "title": "Data Structures",
        "author": "Seymour Lipschutz",
        "category": "Computer Science",
        "quantity": 3
    },
    {
        "id": 103,
        "title": "Operating Systems",
        "author": "Galvin",
        "category": "Computer Science",
        "quantity": 4
    }
]
students = []
issued_books = []
# ------------------- UTILITY FUNCTIONS -------------------
def clear_screen():
    os.system("cls" if os.name == "nt" else "clear")
def pause():
    input("\nPress Enter to continue...")
```

```python
def find_book(book_id):
            return book
    return None
def find_student(student_id):
    for student in students:
        if student["id"] == student_id:
            return student
    return None
# ------------------- BOOK MANAGEMENT -------------------
def add_book():
    clear_screen()
    print("=" * 55)
    print("                ADD NEW BOOK")
    print("=" * 55)
    try:
        book_id = int(input("Enter Book ID: "))
        if find_book(book_id):
            print("\nBook ID already exists!")
            pause()
            return
        title = input("Enter Book Title: ")
        author = input("Enter Author Name: ")
        category = input("Enter Category: ")
        quantity = int(input("Enter Quantity: "))
        if quantity <= 0:
            print("\nQuantity must be greater than 0.")
            pause()
            return
        books.append({
            "id": book_id,
            "title": title,
            "author": author,
            "category": category,
```

```python
            "quantity": quantity
        })
        print("\nBook added successfully!")
    except ValueError:
        print("\nInvalid input!")
    pause()
def display_books():
    clear_screen()
    print("=" * 90)
    print("                              ALL BOOKS")
    print("=" * 90)
    if not books:
        print("No books available.")
        pause()
        return
    print(
        f"{'ID':<8}"
        f"{'Title':<25}"
        f"{'Author':<20}"
        f"{'Category':<20}"
        f"{'Qty':<5}"
    )
    print("-" * 90)
    for book in books:
        print(
            f"{book['id']:<8}"
            f"{book['title'][:24]:<25}"
            f"{book['author'][:19]:<20}"
            f"{book['category'][:19]:<20}"
            f"{book['quantity']:<5}"
        )
    pause()
```

```python
def search_book():
    clear_screen()
    print("=" * 55)
    print("                SEARCH BOOK")
    print("=" * 55)
    keyword = input("Enter title, author or category: ").lower()
    found = False
    print()
    for book in books:
        if (
            keyword in book["title"].lower()
            or keyword in book["author"].lower()
            or keyword in book["category"].lower()
        ):
            print(f"Book ID   : {book['id']}")
            print(f"Title     : {book['title']}")
            print(f"Author    : {book['author']}")
            print(f"Category  : {book['category']}")
            print(f"Available : {book['quantity']}")
            print("-" * 40)
            found = True
    if not found:
        print("No matching book found.")
    pause()
def remove_book():
    clear_screen()
    print("=" * 55)
    print("                REMOVE BOOK")
    print("=" * 55)
    try:
        book_id = int(input("Enter Book ID: "))
        book = find_book(book_id)
        if book is None:
```

```
            print("\nBook not found.")
        elif any(record["book_id"] == book_id for record in issued_books):
            print("\nCannot remove this book because it is issued.")
        else:
            books.remove(book)
            print("\nBook removed successfully.")

    except ValueError:
        print("\nInvalid Book ID.")
    pause()
# ------------------- STUDENT MANAGEMENT -------------------
def add_student():
    clear_screen()
    print("=" * 55)
    print("                ADD STUDENT")
    print("=" * 55)
    try:
        student_id = int(input("Enter Student ID: "))
        if find_student(student_id):
            print("\nStudent ID already exists!")
            pause()
            return
        name = input("Enter Student Name: ")
        course = input("Enter Course: ")
        phone = input("Enter Phone Number: ")
        students.append({
            "id": student_id,
            "name": name,
            "course": course,
            "phone": phone
        })
        print("\nStudent added successfully!")
    except ValueError:
        print("\nInvalid input!")
    pause()
def display_students():
    clear_screen()
    print("=" * 80)
    print("                              ALL STUDENTS")
    print("=" * 80)
    if not students:
        print("No students registered.")
        pause()
        return
    print(
        f"{'ID':<10}"
        f"{'Name':<25}"
        f"{'Course':<25}"
        f"{'Phone':<15}"
    )
    print("-" * 80)
    for student in students:
        print(
            f"{student['id']:<10}"
            f"{student['name'][:24]:<25}"
            f"{student['course'][:24]:<25}"
            f"{student['phone']:<15}"
        )
    pause()
def search_student():
    clear_screen()
    print("=" * 55)
    print("                SEARCH STUDENT")
    print("=" * 55)
    keyword = input("Enter Student ID or Name: ").lower()
    found = False
    for student in students:
        if (
            keyword in str(student["id"])
            or keyword in student["name"].lower()
        ):
            print("\nStudent ID :", student["id"])
            print("Name       :", student["name"])
            print("Course     :", student["course"])
            print("Phone      :", student["phone"])
            print("-" * 40)
            found = True
    if not found:
        print("\nStudent not found.")
    pause()
# -------------------- ISSUE BOOK --------------------
def issue_book():
    clear_screen()
    print("=" * 55)
    print("                  ISSUE BOOK")
    print("=" * 55)
    try:
        student_id = int(input("Enter Student ID: "))
        book_id = int(input("Enter Book ID: "))
        student = find_student(student_id)
        book = find_book(book_id)
        if student is None:
            print("\nStudent not found.")
            pause()
            return
        if book is None:
            print("\nBook not found.")
            pause()
            return
        if book["quantity"] <= 0:
            print("\nBook is currently unavailable.")
            pause()
            return
        # Check whether same student already has the book
        for record in issued_books:
            if (
                record["student_id"] == student_id
                and record["book_id"] == book_id
            ):
                print("\nThis student already has this book.")
                pause()
                return
        issue_date = datetime.now()
        due_date = issue_date + timedelta(days=14)
        issued_books.append({
            "student_id": student_id,
            "book_id": book_id,
            "issue_date": issue_date.strftime("%Y-%m-%d"),
            "due_date": due_date.strftime("%Y-%m-%d")
        })
        book["quantity"] -= 1
        print("\nBook issued successfully!")
        print("Issue Date :", issue_date.strftime("%Y-%m-%d"))
        print("Due Date   :", due_date.strftime("%Y-%m-%d"))
    except ValueError:
        print("\nInvalid input!")
    pause()
# -------------------- RETURN BOOK --------------------
```

```python
# ------------------- RETURN BOOK --------------------
def return_book():
    clear_screen()
    print("=" * 55)
    print("                  RETURN BOOK")
    print("=" * 55)
    try:
        student_id = int(input("Enter Student ID: "))
        book_id = int(input("Enter Book ID: "))
        record = None
        for item in issued_books:
            if (
                item["student_id"] == student_id
                and item["book_id"] == book_id
            ):
                record = item
                break
        if record is None:
            print("\nNo issue record found.")
            pause()
            return
        book = find_book(book_id)
        if book:
            book["quantity"] += 1
        due_date = datetime.strptime(
            record["due_date"],
            "%Y-%m-%d"
        )
        return_date = datetime.now()
        overdue_days = (return_date - due_date).days
        if overdue_days > 0:
            fine = overdue_days * 5
            print(f"\nBook returned late by {overdue_days} day(s).")
            print(f"Fine: Rs. {fine}")
        else:
            print("\nBook returned on time.")
            print("Fine: Rs. 0")
        issued_books.remove(record)
        print("Book returned successfully!")
    except ValueError:
        print("\nInvalid input!")
    pause()
# ------------------- ISSUED BOOKS --------------------
def display_issued_books():
    clear_screen()
    print("=" * 100)
    print("                              ISSUED BOOKS")
    print("=" * 100)
    if not issued_books:
        print("No books are currently issued.")
        pause()
        return
    print(
        f"{'Student ID':<15}"
        f"{'Student Name':<25}"
        f"{'Book ID':<10}"
        f"{'Book Title':<25}"
        f"{'Due Date':<15}"
    )
    print("-" * 100)
    for record in issued_books:
        student = find_student(record["student_id"])
        book = find_book(record["book_id"])
        student_name = student["name"] if student else "Unknown"
        book_title = book["title"] if book else "Unknown"
        print(
            f"{record['student_id']:<15}"
            f"{student_name[:24]:<25}"
            f"{record['book_id']:<10}"
            f"{book_title[:24]:<25}"
            f"{record['due_date']:<15}"
        )
    pause()
# ------------------- STATISTICS --------------------
def library_statistics():
    clear_screen()
    total_books = sum(book["quantity"] for book in books)
    issued_count = len(issued_books)
    total_students = len(students)
    total_book_types = len(books)
    print("=" * 55)
    print("                LIBRARY STATISTICS")
    print("=" * 55)
    print(f"\nDifferent Books   : {total_book_types}")
    print(f"Available Copies : {total_books}")
    print(f"Issued Books     : {issued_count}")
    print(f"Registered Users : {total_students}")
    pause()
# ------------------- MAIN MENU --------------------
def main():
    while True:
        clear_screen()
        print("=" * 65)
        print("              LIBRARY MANAGEMENT SYSTEM")
        print("=" * 65)
        print("\n1.  Add Book")
        print("2.  Display All Books")
        print("3.  Search Book")
        print("4.  Remove Book")
        print("\n5.  Add Student")
        print("6.  Display Students")
        print("7.  Search Student")
        print("\n8.  Issue Book")
        print("9.  Return Book")
        print("10. Display Issued Books")
        print("\n11. Library Statistics")
        print("12. Exit")
        print("=" * 65)
        choice = input("Enter your choice: ")
        if choice == "1":
            add_book()
        elif choice == "2":
            display_books()
        elif choice == "3":
            search_book()
        elif choice == "4":
            remove_book()
        elif choice == "5":
            add_student()
        elif choice == "6":
            display_students()
        elif choice == "7":
            search_student()
        elif choice == "8":
            issue_book()
        elif choice == "9":
            return_book()
        elif choice == "10":
            display_issued_books()
```

```
390             elif choice == "11":
391                 library_statistics()
392             elif choice == "12":
393                 clear_screen()
394                 print("\nThank you for using Library Management System!")
395                 break
396             else:
397                 print("\nInvalid choice!")
398                 pause()
399     # -------------------- PROGRAM START --------------------
400     if __name__ == "__main__":
401         main()
```